As you've probably come to expect, Rails does a good job of providing defaults for layout file locations, but you can override the defaults if you need something different.

Layouts are controller-specific. If the current request is being handled by a controller called *store*, Rails will by default look for a layout called `store` (with the usual `html.erb` or `xml.builder` extension) in the `app/views/layouts` directory. If you create a layout called `application` in the `layouts` directory, it will be applied to all controllers that don't otherwise have a layout defined for them.

You can override this using the `layout` declaration inside a controller. The most basic invocation is to pass it the name of a layout as a string. The following declaration will make the template in the file `standard.html.erb` or `standard.xml.builder` the layout for all actions in the store controller.

The layout file will be looked for in the `app/views/layouts` directory:

```
class StoreController < ApplicationController

  layout "standard"

  # ...
end
```

You can qualify which actions will have the layout applied to them using the `:only` and `:except` qualifiers:

```
class StoreController < ApplicationController

  layout "standard", except: [ :rss, :atom ]

  # ...
end
```

Specifying a layout of `nil` turns off layouts for a controller.

Sometimes you need to change the appearance of a set of pages at runtime. For example, a blogging site might offer a different-looking side menu if the user is logged in, or a store site might have different-looking pages if the site is down